

Project Spec - BoxdIn

Version 1 Review

**Anything in italics is new or changed from the original spec.*

General Description

This app is called BoxdIn, short for "Letterboxd Insights." It is designed for users of Letterboxd, a social networking app for dedicated movie fans. Users can log films in a diary, rate them, write reviews, and engage with a community of fellow film watchers. While Letterboxd's free tier allows full use of these features, most stat-tracking and viewing insights are locked behind a paid subscription. BoxdIn aims to give free users access to a meaningful set of personalized stats and insights at no cost.

Every Letterboxd user profile has a publicly accessible RSS feed. When a user opens BoxdIn and enters their Letterboxd username, the app fetches data from that feed and generates a dashboard of insights about their recent viewing habits. Some additional movie data is pulled from the OMDb API, a free web service that provides information from IMDb.

Data used from the Letterboxd RSS feed:

- Film title
- Watch date
- User star rating
- Film release year
- Rewatch flag (yes/no)

Data used from the OMDb API:

- Genre
- Director
- Runtime
- IMDb rating

BoxdIn displays insights in the following sections:

- Snapshot (Your last 30 days): number of films watched, total cumulative hours watched, most watched genre
- Variety: number of unique genres watched, number of unique directors watched, oldest and newest film watched
- Rating Breakdown: average rating by genre (min. 3 films), average rating by director (min. 3 films), average rating by decade (min. 3 films), overall average difference between your ratings and IMDb ratings, most controversial opinion (the film where your rating differed most from IMDb)

Rating Histogram Chart: this section wasn't in my original plan. It counts how many films fall at each star rating, from 0.5 to 5.0. It came together naturally while I was building the rating breakdown logic, so I added it as its own section.

For version 1, BoxdIn will run as a simple command line app. The user types in a Letterboxd username, hits enter, and the results print out in the terminal. It's not flashy, but it will let me focus on making sure the data and insights pull and populate correctly first. My original plan was that once that's ready, version 2 would have a proper web interface built with Flask, with a text field for the username and a clean layout for results, plus charts like a bar chart of films per decade, a pie chart of genres, a histogram of ratings, and a chart of most watched directors.

**Given the time left in the course, I won't be able to build the Flask interface or any of the other charts. So, the command line version, is going to be my final version for this class.*

Task Vignettes

Vignette 1: Opening the app and entering a username

Chloe is an avid Letterboxd user who logs almost every film she watches. She's curious about her viewing habits but doesn't want to pay for a Letterboxd Pro subscription just to see some basic stats. She opens the app, and is prompted to enter her Letterboxd username. She types it in and hits enter. A moment later her insights are displayed.

Technical details:

- Chloe runs the script from the command line
- App prints a welcome message and prompts for a Letterboxd username
- App always fetches the fresh RSS feed from Letterboxd using the entered username
- The feed is parsed and the relevant data points are pulled out for each film
- App checks if a saved data file already exists for this username
- This part changed from my plan: right now, if a saved file exists, the app loads it and skips OMDb completely. If no saved file exists, it queries OMDb for all 50 films. My original plan was for it to only query OMDb for films not already saved, so it would just fetch what's new. That per-film check isn't built yet, it's still on my TODO list.
- All data is saved to a file so future lookups require fewer OMDb calls

Vignette 2: Missing data

- Chloe tends to log films quickly after watching them, but sometimes she logs a film without leaving a rating. When BoxdIn generates her insights, films without a rating are still counted toward her total films watched and hours watched, but are left out of any calculations that require a star rating. Chloe doesn't notice anything is wrong because the app just works around the gaps.
- Technical details:
- When parsing each RSS entry, app checks whether a rating exists before including it in rating-based calculations
- Films with no rating are still counted for non-rating insights like total films watched, hours watched, genre counts, and director counts

- If OMDb returns incomplete data for a film (e.g. no runtime or no genre), that film is skipped for any insight that requires that field
- App does not crash or display an error, it simply works with whatever data is available

Vignette 3: Invalid username

Chloe wants to show BoxdIn to her friend and types in his Letterboxd username, but makes a typo. The app tries to fetch the RSS feed but comes back empty. Rather than crashing or displaying a confusing error message, BoxdIn lets her know that the username wasn't found and prompts her to try again.

Technical details (this is what I planned, see note below):

- App attempts to fetch the RSS feed using the entered username
- If the feed returns empty or can't be reached, app prints a simple message like "Username not found, please try again"
- App prompts the user to enter a new username rather than closing
- No data is saved for an invalid username

**This issue was not accounted for in the current iteration of the app. It is currently hardcoded for one username to use as an example, but if it wasn't, an invalid username would crash the app.*

Technical Flow

Input

User enters a Letterboxd username as a text string via the command line.

Right now this is hardcoded to a specific username for grading purposes, instead of actually using input(). I'll switch it back to input() before the final version.

Fetch and Parse RSS Feed

App requests the RSS feed from `letterboxd.com/[username]/rss/`. The feed is returned as XML and parsed using `feedparser`. Relevant fields are extracted from each entry: film title, watch date, user rating, release year, rewatch flag. The result is a list of dictionaries, one per film.

Check for Saved Data

App checks if a CSV file already exists for this username.

If the CSV exists, it's loaded and used as-is, no API calls happen at all. If it doesn't exist, OMDb is queried for all 50 films. My original plan was for only new films (ones not already in the CSV) to get sent to OMDb, but that selective, per-film version isn't built yet.

Fetch Data from OMDb

For each new film, the app sends a request to the OMDb API using the film title. OMDb returns JSON containing genre, director, runtime, and IMDb rating. If any of these fields are missing, that film is noted but not discarded entirely.

Something I am aware of as a possible issue is that for films with common or similar titles, the API can occasionally return data for the wrong movie. I haven't fixed this yet

Save Combined Data

RSS data and OMDb data are combined into a pandas dataframe. The dataframe is saved as a CSV file named after the username.

Calculate Insights

The Snapshot section filters for only films watched in the last 30 days before running its calculations. All other sections use the full dataset of up to 50 films. Any insight requiring a rating skips films with no rating. Any insight requiring OMDb data skips films where that data is missing. Rating breakdown sections skip any category with fewer than 3 films.

Display Results

Results are printed to the terminal in the following sections: Snapshot, Variety, and Rating Breakdown. *And also Rating Histogram, since that section got added.*

Technical Feasibility

The RSS feed parsing is the part I'm most confident about. We worked with RSS feeds and feedparser in HCI 574, so I know that piece works.

The OMDb API queries and CSV caching were things I hadn't tried before starting this project. Your example code for querying OMDb gave me a good starting point. The main unknown was how reliably OMDb returns clean data for all 50 films and how long 50 sequential requests would take to run.

Self Assessment

The biggest change from my original sketch wasn't really a change. I had identified the API and the general concept, but I hadn't thought through what insights to actually show. Working through the spec, I ended up with a lot of ideas, and I enjoyed brainstorming different ways to give new context to the data. I ended up scrapping several ideas that weren't interesting enough or didn't fit well with the others. I'm happy with the list I settled on.

I've never done most of what this app requires, but I think it's all achievable. I can picture how it fits together in my head, which feels like a good sign. The biggest challenge will probably just be the sheer amount of work. There are a lot of individual insights to build out, and each one needs to work correctly and handle missing data properly. It's not that any single piece seems impossible, it's just that there's a lot of it.

The part I'm least confident about is working with the OMDb API. Making sure the data comes back correctly for all 50 films and gets stored and displayed properly is where I'd expect to hit a few potholes and have to iron out some kinks.